

Disorder Potential and the Cost of Search

A balance law and a separation by disorder location

Jonathan R. Landers

October 4, 2026

Abstract. Sorting creates structure that makes subsequent search cheaper. We study this connection using the squared rank-displacement potential from the permutohedral view of sorting. Given a potential budget E , the optimal worst-case comparison-search cost is $\Theta(\log n + \min\{n, \sqrt{E}\})$. A matching lower bound explains the square root: a target can hide among r positions while using only $\Theta(r^2)$ potential. We then construct two promised input families with identical maximum potential but different optimal search costs. A potential balance accounts exactly for disorder removed; the arrangement of the remaining disorder determines whether search can improve on the guarantee obtained from the scalar budget alone.

1 From preprocessing to usable structure

An unordered array can require n comparisons to search. Once sorted, it supports binary search in $\Theta(\log n)$ comparisons. Producing that order by comparison sorting requires $\Theta(n \log n)$ comparisons in the unrestricted worst case. These bounds suggest an accounting question: how does the structure created during preprocessing translate into the work saved afterward?

Actual effort cannot itself be the conserved quantity. Two sorting procedures can spend very different amounts of work and produce the same array. Existing order also matters: an almost-sorted input may need little additional rearrangement. We therefore account for *remaining disorder*, rather than identify all work spent with useful progress. The potential in Landers [1] supplies a geometric measure. Our aim is to connect that measure to search complexity, then determine what its scalar value leaves out.

2 Model and geometric potential

Let $A = (a_1, \dots, a_n)$ contain $n \geq 2$ distinct keys from an arbitrary totally ordered universe. Put $x_i = \text{rank}_A(a_i)$ and $v_s = (1, \dots, n)$. The rank word x is a vertex of the permutohedron $\text{conv}\{\pi(v_s) : \pi \in S_n\}$. Define

$$V(A) = \frac{1}{2} \|x - v_s\|_2^2 = \frac{1}{2} \sum_{i=1}^n (x_i - i)^2, \quad d(A) = \max_i |x_i - i|. \quad (1)$$

Thus V measures squared Euclidean distance to the sorted vertex; it is not the adjacent-swap graph distance. The ranks are an analytical description, not an oracle available to the search algorithm.

A deterministic search algorithm queries indices i , receiving the outcome of $y < a_i$, $y = a_i$, or $y > a_i$. It must return the index of y , or report absence. There is no auxiliary index or preprocessing transcript. A numerical promise $V(A) \leq E$ is supplied; obtaining that promise is a separate preprocessing cost. With $C_S(A, y; E)$ denoting the number of queries, define

$$Q^*(n, E) = \inf_{S \text{ correct}} \sup_{A: V(A) \leq E} \sup_y C_S(A, y; E). \quad (2)$$

All logarithms in complexity expressions are base 2; constants are independent of n, E .

3 How potential limits displacement

Lemma 1. *For every rank word, $V(A) \geq d(A)(d(A) + 1)/2$. Consequently, if $V(A) \leq E$, then*

$$d(A) \leq D_E := \min \left\{ n - 1, \left\lceil \frac{\sqrt{1 + 8E} - 1}{2} \right\rceil \right\}. \quad (3)$$

Proof. Set $\delta_i = x_i - i$ and select j with $|\delta_j| = d$. Since $\sum_i \delta_i = 0$, $\sum_{i \neq j} |\delta_i| \geq d$. Integer displacements satisfy $\delta_i^2 \geq |\delta_i|$, so $2V = d^2 + \sum_{i \neq j} \delta_i^2 \geq d^2 + d$. Solving this inequality gives (3). $\square$

4 A tight search bound from potential alone

Theorem 2. For $n \geq 2$ and $E \geq 0$, the model in Section 2 satisfies

$$Q^*(n, E) = \Theta\left(\log n + \min\{n, \sqrt{E}\}\right). \quad (4)$$

Proof. Upper bound. Suppose first that $d(A) \leq D$ is known. Maintain an interval $[L, H]$ that contains the target whenever it is present. If its length exceeds $8D + 4$, query $m = \lfloor (L + H)/2 \rfloor$. Return m on equality. Otherwise update

$$\begin{aligned} a_m < y : \quad L &\leftarrow \max\{L, m - 2D + 1\}, \\ a_m > y : \quad H &\leftarrow \min\{H, m + 2D - 1\}. \end{aligned} \quad (5)$$

Once the interval has length at most $8D + 4$, scan it and return the target's index, or report absence.

To prove the updates safe, suppose $y = a_p$ and let $r = x_p$ be its rank. If $a_m < y$, then

$$p \geq r - D \geq x_m + 1 - D \geq m - 2D + 1. \quad (6)$$

If $a_m > y$, similarly $p \leq r + D \leq x_m - 1 + D \leq m + 2D - 1$. Thus the target is never discarded. If absent, no equality is returned, and the final scan correctly reports absence.

For interval length N , either update leaves at most $\lfloor N/2 \rfloor + 2D$ candidates. When $N > 8D + 4$, this is less than $3N/4$. There are therefore $O(\log n)$ midpoint queries, followed by $O(\min\{n, D + 1\})$ scan queries. Taking $D = D_E \leq \sqrt{2E}$ proves the upper bound in (4). This is a direct potential-based formulation of bounded-displacement search [2].

Lower bound. Fix ordered keys $a_1 < \dots < a_n$ and a target $y < a_1$. Let $A^{(0)} = (a_1, \dots, a_n)$, in which y is absent. For each j , let $A^{(j)}$ be the same array with entry j replaced by y . Its rank word is

$$x^{(j)} = (2, 3, \dots, j, 1, j + 1, \dots, n), \quad V(A^{(j)}) = \frac{j(j-1)}{2}. \quad (7)$$

For $j = 1$ the first segment is empty and the rank word is the identity. Set

$$r_E = \min\left\{n, \left\lfloor \frac{1 + \sqrt{1 + 8E}}{2} \right\rfloor\right\} = D_E + 1. \quad (8)$$

All $A^{(j)}$ with $1 \leq j \leq r_E$ satisfy $V \leq E$. On $A^{(0)}$, a correct algorithm must query every one of these positions. Otherwise an unqueried j gives the same entire transcript on $A^{(j)}$, yet requires a different answer. Hence at least r_E queries are necessary, and $r_E = \Theta(1 + \min\{n, \sqrt{E}\})$.

Independently, sorted arrays belong to the promised class. For a fixed sorted array, the search decision tree must distinguish n matching indices and absence. Each query has at most three outcomes, so its worst-case depth is at least $\log_3(n + 1) = \Omega(\log n)$. The larger of the two lower bounds is at least half their sum. This proves (4). $\square$

Why a square root appears. In (7), the minimum moves $j - 1$ positions, while the $j - 1$ preceding entries each shift by one. Its large displacement contributes $(j - 1)^2/2$ to the potential, and the smaller shifts contribute $(j - 1)/2$. Thus $\Theta(r^2)$ potential permits a target to hide among r candidate positions. The square root is a geometric obstruction to search, not an assumption about a particular sorting algorithm.

5 Equal potential budgets, different searchable structure

Theorem 2 is tight when only E is supplied. A more descriptive promise can permit substantially faster search. The following separation makes that distinction explicit; it compares maxima over families, not equal-potential individual inputs.

Proposition 3. *Let $r \geq 2$, $E = r(r-1)/2$, and $n \geq 2E$. There are two known families of length- n arrays with maximum potential exactly E , whose optimal worst-case search costs are respectively $\Theta(\log n + r)$ and $\Theta(\log n)$.*

Proof. Define $\mathcal{C}_{\text{conc}}$ by taking an arbitrary sorted array, removing its minimum, and inserting it at a position $j \in \{1, \dots, r\}$. Its rank words are precisely those in (7), so its maximum potential is E . Search by scanning the first r entries, then binary-searching the sorted suffix if necessary. This uses $O(r + \log n)$ queries. Every array used in the hiding argument belongs to $\mathcal{C}_{\text{conc}}$, including $A^{(0)}$. That argument supplies $\Omega(r)$, and the sorted subfamily supplies $\Omega(\log n)$, proving the matching bound.

Define $\mathcal{C}_{\text{spread}}$ by taking an arbitrary sorted array and independently swapping any subset of the E fixed pairs $(1, 2), (3, 4), \dots, (2E-1, 2E)$. Each swapped pair contributes $(1^2 + (-1)^2)/2 = 1$ to V . Swapping every pair gives $V = E$, but every array has $d \leq 1$. The search procedure (5) with $D = 1$ uses $O(\log n)$ queries. The sorted subfamily gives the matching lower bound. $\square$

Known arrangement of disorder	Maximum V	Optimal search
One minimum moved within a prefix of length r	E	$\Theta(\log n + \sqrt{E})$
Many independent adjacent-pair swaps	E	$\Theta(\log n)$

For $n = 2E = r(r-1)$ and $r \rightarrow \infty$, these costs become $\Theta(\sqrt{E})$ and $\Theta(\log E)$. The same total potential can describe one deeply displaced item or many shallow displacements. The family promise tells search which geometry is possible; the scalar budget alone does not.

6 The balance law and its computational meaning

For a preprocessing trajectory $x^{(0)}, \dots, x^{(B)}$, let $\Delta_k = V(x^{(k)}) - V(x^{(k+1)})$ and $A_B = \sum_{k=0}^{B-1} \Delta_k$. Telescoping gives

$$\boxed{V(x^{(0)}) = A_B + V(x^{(B)})}. \quad (9)$$

For inversion-removing adjacent swaps, $\Delta_k > 0$; swapping adjacent ranks $a > b$ gives $\Delta_k = a - b$ [1]. In the continuous relaxation $\dot{x} = -\nabla V$, differentiation gives $\dot{V} = -\|\nabla V\|_2^2$, and integration yields

$$V(x(0)) = V(x(t)) + \int_0^t \|\nabla V(x(s))\|_2^2 ds. \quad (10)$$

Thus initial disorder equals residual disorder plus accumulated decrease. If $V(x^{(0)}) \leq E_0$ and preprocessing certifies a decrease of at least a , then $V(x^{(B)}) \leq E_0 - a$. Theorem 2 gives the tight guarantee from this numerical promise alone, with $E = E_0 - a$. Proposition 3 shows why additional location information can improve it.

This is an exact *potential balance* and a sharp *search guarantee*, not an equality between elapsed computation and savings. B counts trajectory steps, not all comparisons needed to choose or certify them. With preprocessing cost C_P and q subsequent searches, total cost is $C_P + \sum_{j=1}^q C_S(A', y_j)$. The further question is how cheaply preprocessing can produce both low residual potential and useful structural certificates. The note connects established adaptive search machinery [2] to the geometric potential; it makes no claim of priority.

References

- [1] J. R. Landers. *Sorting as Gradient Flow on the Permutohedron*. arXiv:2504.16706v2, 2026.
- [2] Y. Disser and S. Kratsch. *Robust and Adaptive Search*. STACS 2017, LIPIcs 66, 26:1–26:14. doi:10.4230/LIPIcs.STACS.2017.26.